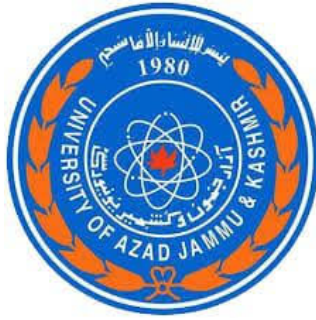




UNIVERSITY OF AZAD JAMMU & KASHMIR

MUZAFFARABAD, AZAD KASHMIR

DIGITAL CLASSROOM MANAGEMENT SYSTEM

C++ Object-Oriented Programming Project Report

Second Semester University Project

Department of Computer Science

Project Title	Digital Classroom Management System
Programming Language	C++
Programming Paradigm	Object-Oriented Programming (OOP)
Semester	2nd Semester

Prepared for: C++ / Object-Oriented Programming Course

Table of Contents

1	Introduction
2	Project Overview
3	Problem Statement
4	Objectives
5	System Design and Classes
6	C++ OOP Concepts Used
7	Detailed Code Explanation
8	Runtime Polymorphism and Dynamic Binding
9	Authentication and Role Handling
10	Course Management
11	Program Flow
12	Testing and Expected Output
13	Advantages

1. Introduction

The Digital Classroom Management System is a C++ project designed to demonstrate core Object-Oriented Programming concepts through a simple academic management scenario. The supplied implementation models users, students, teachers, and courses using classes, inheritance, virtual functions, constructors, access control, and runtime polymorphism.

This report is based specifically on the provided C++ source code and describes the OOP concepts demonstrated in that source.

2. Project Overview

Project	Digital Classroom Management System
Language	C++
Main Header Files	iostream, string, vector
Main Classes	User, Student, Teacher, Course
Inheritance	Student and Teacher inherit from User
Polymorphism	Virtual showDashboard() overridden by derived classes
Data Structure	vector<Course>
Authentication	Username, password and role selection
Execution	Console-based C++ program

3. Problem Statement

A classroom environment contains different types of users and academic information. A student and a teacher do not perform the same tasks, so the program needs a structure that can represent common user information while allowing role-specific behavior. The project addresses this by creating a base User class and specialized Student and Teacher classes, while a Course class represents academic courses.

4. Objectives

- Demonstrate the fundamental principles of C++ Object-Oriented Programming.
- Create reusable classes for users and courses.
- Use inheritance to represent specialized user roles.
- Use virtual functions and overriding for runtime polymorphism.
- Apply constructors to initialize objects.
- Apply encapsulation through protected/private data members.
- Use vector to store and process multiple Course objects.
- Implement a simple role-based login flow.

5. System Design and Classes

5.1 User Base Class

User is the base class. It stores the common attributes name and role. Its constructor receives these values and initializes the object. The class also provides a virtual showDashboard() function and a virtual destructor.

```
class User
{
protected:
    string name;
    string role;

public:
    User(string n, string r) {
        name = n;
        role = r;
    }

    virtual void showDashboard() {
        cout << "\nWelcome to Digital Classroom\n";
    }

    virtual ~User() {}
};
```

5.2 Student Class

Student publicly inherits from User. Its constructor calls the base-class constructor with the role "Student". It overrides showDashboard() and provides student-specific options.

```
class Student : public User
{
public:
    Student(string n) : User(n, "Student") {}

    void showDashboard() override {
        cout << "\n===== STUDENT DASHBOARD =====\n";
    }
};
```

5.3 Teacher Class

Teacher also publicly inherits from User. Its constructor initializes the role as Teacher and its overridden dashboard contains teacher-specific management options.

```
class Teacher : public User
{
public:
    Teacher(string n) : User(n, "Teacher") {}

    void showDashboard() override {
        cout << "\n===== TEACHER DASHBOARD =====\n";
    }
};
```

5.4 Course Class

Course is an independent class that represents an academic course. Its private members are code, title and teacher. The constructor initializes these values and display() prints the course information.

```
class Course
{
private:
```

```

    string code;
    string title;
    string teacher;

public:
    Course(string c, string t, string tr) {
        code = c;
        title = t;
        teacher = tr;
    }

    void display() {
        cout << code << " | "
              << title << " | "
              << teacher << endl;
    }
};

```

6. C++ OOP Concepts Used

6.1 Classes and Objects

A class is a blueprint for creating objects. The project defines User, Student, Teacher and Course classes. Objects are created from these classes.

6.2 Encapsulation

Encapsulation means keeping data inside a class and controlling access to it through the class interface. Course demonstrates this with private code, title and teacher members. User uses protected members for access by derived classes.

6.3 Inheritance

Inheritance allows a new class to reuse features of an existing class. Student : public User and Teacher : public User show inheritance from the User base class.

6.4 Polymorphism

Polymorphism allows the same interface to produce different behavior. showDashboard() is virtual in User and overridden in Student and Teacher.

6.5 Abstraction

The User class provides a common interface for dashboard behavior without requiring main() to know every implementation detail. User is not abstract because showDashboard() has a body.

6.6 Constructors

Constructors initialize objects. User, Student, Teacher and Course each use constructors. Student and Teacher use initializer lists for their base User object.

6.7 Function Overriding

Student and Teacher define showDashboard() with the same signature as the virtual function in User. The override keyword confirms the overriding relationship.

6.8 Virtual Functions

User declares showDashboard() as virtual, enabling dynamic dispatch. The User destructor is also virtual, which is appropriate when deleting derived objects through a base-class pointer.

6.9 Access Specifiers

The project uses public, protected and private access. User data is protected for derived classes, while Course data members are private.

6.10 Collection of Objects

The main function creates a vector containing multiple Course objects.

6.11 Dynamic Memory

Student or Teacher objects are created with new and stored in a User* pointer. delete currentUser releases the allocated object.

7. Detailed Code Explanation

7.1 Header Files

The program includes iostream for console input/output, string for text data, and vector for storing multiple Course objects.

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
```

7.2 Base-Class Initialization

The User constructor receives a name and role and assigns them to the corresponding members. This gives every User-derived object common identity information.

7.3 Runtime Polymorphism

```
User* currentUser = nullptr;

if (roleChoice == 1 &&
    username == "student" &&
    password == "student123")
{
    currentUser = new Student("Student");
}
else if (roleChoice == 2 &&
         username == "teacher" &&
         password == "teacher123")
{
    currentUser = new Teacher("Teacher");
}

currentUser->showDashboard();
```

The pointer type is User*, but the object can be Student or Teacher. Because showDashboard() is virtual, the correct overridden function is selected at runtime.

8. Runtime Polymorphism and Dynamic Binding

The program demonstrates dynamic binding by declaring a User pointer, creating a Student or Teacher object based on authentication, assigning the derived object to the base pointer, and calling

currentUser->showDashboard()). The actual object type determines which dashboard function executes.

Pointer Type	Actual Object	Function Called
User*	Student	Student::showDashboard()
User*	Teacher	Teacher::showDashboard()

9. Authentication and Role Handling

The main function asks the user for a username, password and role choice. The supplied source contains two valid combinations: student/student123 for Student and teacher/teacher123 for Teacher. If the values do not match, the program prints an invalid-login message and terminates.

Role	Username	Password
Student	student	student123
Teacher	teacher	teacher123

Note: these credentials are hard-coded in the supplied educational source and are suitable for demonstration only.

10. Course Management

The project creates a vector of Course objects. Five sample courses are added: CS-201 Object Oriented Programming, CS-202 Data Structures and Algorithms, CS-203 Database Systems, CS-204 Digital Logic Design, and CS-205 Computer Organization. A range-based for loop then calls display() for each course.

```
vector<Course> courses =
{
    Course("CS-201", "Object Oriented Programming", "Dr. Ahmed"),
    Course("CS-202", "Data Structures and Algorithms", "Dr. Ali"),
    Course("CS-203", "Database Systems", "Dr. Hamza"),
    Course("CS-204", "Digital Logic Design", "Dr. Usman"),
    Course("CS-205", "Computer Organization", "Dr. Bilal")
};

for (Course &course : courses)
{
    course.display();
}
```

11. Program Flow

1. Program starts in main().
2. System title is displayed.
3. Username is entered.
4. Password is entered.
5. User selects Student or Teacher role.
6. Credentials and role are checked.
7. A Student or Teacher object is created through a User pointer.

8. Runtime polymorphism displays the appropriate dashboard.
9. Course objects are created and displayed.
10. System status is displayed.
11. The dynamically allocated user object is deleted.
12. Program terminates.

12. Testing and Expected Output

Test Case	Input / Condition	Expected Result
Student login	student / student123 / Student	Student dashboard displayed
Teacher login	teacher / teacher123 / Teacher	Teacher dashboard displayed
Invalid login	Wrong username/password	Invalid credentials message
Course display	Successful login	Five courses displayed
Polymorphism	Student or Teacher object via User*	Correct overridden dashboard runs

13. Advantages

- Clear demonstration of major introductory C++ OOP concepts.
- Reusable User base class reduces duplication between Student and Teacher.
- Runtime polymorphism makes role-specific dashboard behavior possible.
- Course objects provide a clean model for academic information.
- The program is small enough to understand and explain in a second-semester viva.